

=====

## COMPTE RENDU - SOLVEUR INVERSE

=====

### 1. OBJECTIF

Le but du projet est d'optimiser une grille de lettres afin de maximiser le nombre de mots trouvés par le solveur direct.

Le score utilisé est :

score = somme(longueur\_carrée des mots)

Les mots longs donnent donc un meilleur score. Plusieurs petits mots ne donnent pas forcément un meilleur score que quelques mot long.

-----

### 2. architecture

Le programme contient :

- un solveur direct :  
recherche les mots dans la grille
- un solveur inverse :  
modifie la grille pour améliorer le score

Fonctions principales :

- evaluate\_inverse\_solver()
- run\_baseline\_inverse\_solver()
- run\_my\_heuristic\_inverse\_solver()

-----

### 3. data/données

Le fichier :  
data/letters\_freq.txt

contient les fréquences des lettres utilisées par l'heuristique améliorée.

Exemple :

E 12.7  
A 8.1  
...

#### 4. Algo

##### A. Baseline

- modification aléatoire d'une lettre
- conservation uniquement si le score augmente

##### B. Heuristique améliorée

- initialisation avec fréquences des lettres
- mutations pondérées par fréquence

-----

#### 5. Resultats

L'heuristique améliorée obtient généralement :

- plus de mots trouvés
- un meilleur score
- une convergence plus rapide

-----

#### 6. Avantages et inconvénients de ma méthode

Avantages :

- simple
- rapide
- améliore les résultats

Inconvénients :

- peut rester bloqué dans un optimum local
- dépend des fréquences des lettres

-----

#### 7. End

L'utilisation des fréquences des lettres améliore les performances du solveur inverse par rapport à la méthode baseline.

=====